```
import os

for file in os.listdir("/content"):
    if "KDD" in file.upper():
        print(file)
```

Show hidden output

```
import pandas as pd

columns = [
    "duration", "protocol_type", "service", "flag", "src_bytes",
    "dst_bytes", "land", "wrong_fragment", "urgent", "hot",
    "num_failed_logins", "logged_in", "num_compromised",
    "root_shell", "su_attempted", "num_root", "num_file_creations",
    "num_shells", "num_access_files", "num_outbound_cmds",
    "is_host_login", "is_guest_login", "count", "srv_count",
    "serror_rate", "srv_serror_rate", "rerror_rate",
    "srv_rerror_rate", "same_srv_rate", "diff_srv_rate",
    "srv_diff_host_rate", "dst_host_count", "dst_host_srv_count",
    "dst_host_same_srv_rate", "dst_host_diff_srv_rate",
    "dst_host_same_src_port_rate", "dst_host_srv_diff_host_rate",
    "dst_host_serror_rate", "dst_host_srv_serror_rate",
    "dst_host_rerror_rate", "dst_host_srv_rerror_rate",
    "label", "difficulty"
]

train_path = "/content/KDDTrain+.txt"
test_path = "/content/KDDTest+.txt"

train_df = pd.read_csv(train_path, names=columns)
test_df = pd.read_csv(test_path, names=columns)

print("Training shape:", train_df.shape)
print("Testing shape:", test_df.shape)

print("\nTraining sample:")
display(train_df.head())

print("\nTraining labels:")
print(train_df["label"].value_counts().head(10))
```

```
Training shape: (125973, 43)
Testing shape: (22544, 43)

Training sample:
```

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	...	dst_host_same_srv_ra
0	0	tcp	ftp_data	SF	491	0	0	0	0	0	...	0.
1	0	udp	other	SF	146	0	0	0	0	0	...	0.
2	0	tcp	private	S0	0	0	0	0	0	0	...	0.
3	0	tcp	http	SF	232	8153	0	0	0	0	...	1.
4	0	tcp	http	SF	199	420	0	0	0	0	...	1.

5 rows × 43 columns

```
Training labels:
label
normal         67343
neptune        41214
satan           3633
ipsweep         3599
portsweep       2931
smurf           2646
nmap            1493
back             956
teardrop         892
warezclient      890
Name: count, dtype: int64
```

```
print("TRAINING LABEL DISTRIBUTION")
train_counts = train_df["label"].value_counts()
train_summary = pd.DataFrame({
    "count": train_counts,
    "percentage": (train_counts / len(train_df) * 100).round(2)
})
display(train_summary)
```

```
print("\nTESTING LABEL DISTRIBUTION")
test_counts = test_df["label"].value_counts()
test_summary = pd.DataFrame({
    "count": test_counts,
    "percentage": (test_counts / len(test_df) * 100).round(2)
})
display(test_summary)
```

Next steps: Generate code with `train_summary` New interactive sheet Generate code with `test_summary` New interactive sheet

TRAINING LABEL DISTRIBUTION

label	count	percentage
normal	67343	53.46
neptune	41214	32.72
satan	3633	2.88
ipsweep	3599	2.86
portsweep	2931	2.33
smurf	2646	2.10
nmap	1493	1.19
back	956	0.76
teardrop	892	0.71
warezclient	890	0.71
pod	201	0.16
guess_passwd	53	0.04
buffer_overflow	30	0.02
warezmaster	20	0.02

```
train_labels = set(train_df["label"].unique())
test_labels = set(test_df["label"].unique())

unseen_labels = sorted(test_labels - train_labels)

print("Attack/label classes in training:", len(train_labels))
print("Classes in testing:", len(test_labels))
print("Test classes unseen during training:", len(unseen_labels))
print(unseen_labels)
```

```
es in training: 23
: 38
n during training: 17
unnel', 'mailbomb', 'mscan', 'named', 'processtable', 'ps', 'saint', 'sendmail', 'snmpgetattack', 'snmpguess', 'sqlattack',
```

phf	4	0.00
perl	3	0.00
spy	2	0.00

TESTING LABEL DISTRIBUTION

```
import pandas as pd
from sklearn.preprocessing import OrdinalEncoder, LabelEncoder

# Separate features and labels
X_train = train_df.drop(columns=["label", "difficulty"])
X_test = test_df.drop(columns=["label", "difficulty"])

# Encode categorical features using training data only
cat_cols = ["protocol_type", "service", "flag"]
encoder = OrdinalEncoder(
    handle_unknown="use_encoded_value",
    unknown_value=-1
)

X_train[cat_cols] = encoder.fit_transform(X_train[cat_cols])
X_test[cat_cols] = encoder.transform(X_test[cat_cols])

# Binary targets: normal = 0, attack = 1
y_train_binary = (train_df["label"] != "normal").astype(int)
y_test_binary = (test_df["label"] != "normal").astype(int)

# Multiclass targets: fit classes on training labels only
label_encoder = LabelEncoder()
y_train_multi = label_encoder.fit_transform(train_df["label"])

# Mark test labels unseen during training as -1
test_label_map = {
    label: idx for idx, label in enumerate(label_encoder.classes_)
}
y_test_multi = test_df["label"].map(test_label_map).fillna(-1).astype(int)

print("Feature shapes:", X_train.shape, X_test.shape)
print("Binary training labels:\n", y_train_binary.value_counts())
print("Known multiclass labels:", len(label_encoder.classes_))
print("Unseen multiclass test rows:", (y_test_multi == -1).sum())
```

```
Feature shapes: (125973, 41) (22544, 41)
Binary training labels:
```

multihop	18	0.08

```
 label
0    67343
1    58630
Name: count, dtype: int64
Known multiclass labels: 23
Unseen multiclass test rows: 3750
```

named	17	0.08
ps	15	0.07
sendmail	14	0.06
xterm	13	0.06
phf	2	0.01
imap	1	0.00

```
%pip -q install xgboost

import time
import pandas as pd

from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix
)
from xgboost import XGBClassifier

print("Random Forest and XGBoost imported successfully.")
```

```
Random Forest and XGBoost imported successfully.
```

```
import pandas as pd

train_path = "/content/KDDTrain+.txt"
test_path = "/content/KDDTest+.txt"

columns = [
    "duration", "protocol_type", "service", "flag", "src_bytes",
    "dst_bytes", "land", "wrong_fragment", "urgent", "hot",
    "num_failed_logins", "logged_in", "num_compromised",
    "root_shell", "su_attempted", "num_root", "num_file_creations",
    "num_shells", "num_access_files", "num_outbound_cmds",
    "is_host_login", "is_guest_login", "count", "srv_count",
    "serror_rate", "srv_serror_rate", "rerror_rate",
    "srv_rerror_rate", "same_srv_rate", "diff_srv_rate",
    "srv_diff_host_rate", "dst_host_count", "dst_host_srv_count",
    "dst_host_same_srv_rate", "dst_host_diff_srv_rate",
    "dst_host_same_src_port_rate", "dst_host_srv_diff_host_rate",
    "dst_host_serror_rate", "dst_host_srv_serror_rate",
    "dst_host_rerror_rate", "dst_host_srv_rerror_rate",
    "label", "difficulty"
]

train_df = pd.read_csv(train_path, names=columns)
test_df = pd.read_csv(test_path, names=columns)

print("Training dataset:", train_df.shape)
print("Testing dataset:", test_df.shape)
print("\nTraining data preview:")
display(train_df.head())
```

```
Training dataset: (125973, 43)
Testing dataset: (22544, 43)

Training data preview:
```

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	...	dst_host_same_srv_ra
0	0	tcp	ftp_data	SF	491	0	0	0	0	0	...	0.
1	0	udp	other	SF	146	0	0	0	0	0	...	0.
2	0	tcp	private	S0	0	0	0	0	0	0	...	0.
3	0	tcp	http	SF	232	8153	0	0	0	0	...	1.
4	0	tcp	http	SF	199	420	0	0	0	0	...	1.

5 rows × 43 columns

```
import os

print(os.listdir("/content"))
```

```
['.config', 'KDDTest+.txt', 'KDDTrain+.txt', 'sample_data']
```

```
from sklearn.preprocessing import OrdinalEncoder, LabelEncoder

# Separate input features; exclude target and difficulty columns
X_train = train_df.drop(columns=["label", "difficulty"]).copy()
X_test = test_df.drop(columns=["label", "difficulty"]).copy()

# Encode categorical input features
cat_cols = ["protocol_type", "service", "flag"]

encoder = OrdinalEncoder(
    handle_unknown="use_encoded_value",
    unknown_value=-1
)

X_train[cat_cols] = encoder.fit_transform(X_train[cat_cols])
X_test[cat_cols] = encoder.transform(X_test[cat_cols])

# Binary targets: 0 = Normal, 1 = Attack
y_train_binary = (train_df["label"] != "normal").astype(int)
y_test_binary = (test_df["label"] != "normal").astype(int)

# Multiclass targets: preserve attack category labels
label_encoder = LabelEncoder()
y_train_multi = label_encoder.fit_transform(train_df["label"])

# Test labels absent from training are marked -1
test_label_map = {
    label: idx for idx, label in enumerate(label_encoder.classes_)
}
y_test_multi = (
    test_df["label"].map(test_label_map).fillna(-1).astype(int)
)

print("X_train:", X_train.shape)
print("X_test:", X_test.shape)
print("\nBinary training targets:")
print(y_train_binary.value_counts().sort_index().rename(index={0: "Normal (0)", 1: "Attack (1)"}))
print("\nNumber of multiclass labels in training:", len(label_encoder.classes_))
print("Unseen multiclass test rows:", (y_test_multi == -1).sum())
```

```
X_train: (125973, 41)
X_test: (22544, 41)

Binary training targets:
label
Normal (0)    67343
Attack (1)    58630
Name: count, dtype: int64

Number of multiclass labels in training: 23
Unseen multiclass test rows: 3750
```

```
import time
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    classification_report, confusion_matrix
)
from xgboost import XGBClassifier

models = {
    "Random Forest": RandomForestClassifier(
        n_estimators=100,
        random_state=42,
        n_jobs=-1
    ),
    "XGBoost": XGBClassifier(
        n_estimators=100,
        max_depth=6,
        learning_rate=0.1,
        subsample=0.8,
        colsample_bytree=0.8,
        objective="binary:logistic",
        eval_metric="logloss",
        tree_method="hist",
        random_state=42,
        n_jobs=-1
    )
}

results = []
trained_binary_models = {}
```

```
for name, model in models.items():
    print(f"\nTraining {name}...")
    start = time.time()

    model.fit(X_train, y_train_binary)
    training_time = time.time() - start

    predictions = model.predict(X_test)
    trained_binary_models[name] = model

    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test_binary, predictions),
        "Precision": precision_score(y_test_binary, predictions, zero_division=0),
        "Recall": recall_score(y_test_binary, predictions, zero_division=0),
        "F1 Score": f1_score(y_test_binary, predictions, zero_division=0),
        "Training Time (sec)": round(training_time, 2)
    })

    print(f"Training time: {training_time:.2f} seconds")
    print("\nClassification Report:")
    print(classification_report(
        y_test_binary,
        predictions,
        target_names=["Normal", "Attack"],
        zero_division=0
    ))

    print("Confusion Matrix:")
    print(confusion_matrix(y_test_binary, predictions))

binary_results = pd.DataFrame(results).set_index("Model")
print("\nFinal Binary Classification Comparison:")
display(binary_results.round(4))
```

```
Training Random Forest...
Training time: 7.12 seconds

Classification Report:
              precision    recall  f1-score   support

      Normal       0.66      0.97      0.78      9711
      Attack       0.97      0.62      0.75     12833

    accuracy                           0.77     22544
   macro avg       0.81      0.80      0.77     22544
weighted avg       0.83      0.77      0.77     22544

Confusion Matrix:
[[9434  277]
 [4893 7940]]

Training XGBoost...
Training time: 2.17 seconds

Classification Report:
              precision    recall  f1-score   support

      Normal       0.69      0.97      0.81      9711
      Attack       0.97      0.67      0.79     12833

    accuracy                           0.80     22544
   macro avg       0.83      0.82      0.80     22544
weighted avg       0.85      0.80      0.80     22544

Confusion Matrix:
[[9430  281]
 [4244 8589]]

Final Binary Classification Comparison:
```

Model	Accuracy	Precision	Recall	F1 Score	Training Time (sec)
Random Forest	0.7707	0.9663	0.6187	0.7544	7.12
XGBoost	0.7993	0.9683	0.6693	0.7915	2.17

```
#Now Multiclass classification
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
import pandas as pd
```

```
import time

# Keep only test rows with labels seen during training
known_mask = y_test_multi != -1
X_test_known = X_test.loc[known_mask]
y_test_multi_known = y_test_multi.loc[known_mask]

multi_models = {
    "Random Forest": RandomForestClassifier(
        n_estimators=100,
        random_state=42,
        n_jobs=-1
    ),
    "XGBoost": XGBClassifier(
        n_estimators=100,
        max_depth=6,
        learning_rate=0.1,
        subsample=0.8,
        colsample_bytree=0.8,
        objective="multi:softprob",
        num_class=len(label_encoder.classes_),
        eval_metric="mlogloss",
        tree_method="hist",
        random_state=42,
        n_jobs=-1
    )
}

multi_results = []

for name, model in multi_models.items():
    print(f"\nTraining {name} for multiclass classification...")
    start = time.time()

    model.fit(X_train, y_train_multi)
    training_time = time.time() - start

    predictions = model.predict(X_test_known)

    multi_results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test_multi_known, predictions),
        "Macro Precision": precision_score(
            y_test_multi_known, predictions, average="macro", zero_division=0
        ),
        "Macro Recall": recall_score(
            y_test_multi_known, predictions, average="macro", zero_division=0
        ),
        "Macro F1": f1_score(
            y_test_multi_known, predictions, average="macro", zero_division=0
        ),
        "Training Time (sec)": round(training_time, 2)
    })

    print(f"Training time: {training_time:.2f} seconds")

print("\nTest rows with known classes:", len(y_test_multi_known))
print("Excluded test rows with unseen classes:", int((~known_mask).sum()))

multi_results_df = pd.DataFrame(multi_results).set_index("Model")
display(multi_results_df.round(4))
```

```
Training Random Forest for multiclass classification...
Training time: 7.67 seconds

Training XGBoost for multiclass classification...
Training time: 28.41 seconds

Test rows with known classes: 18794
Excluded test rows with unseen classes: 3750
```

Model	Accuracy	Macro Precision	Macro Recall	Macro F1	Training Time (sec)
Random Forest	0.8662	0.5153	0.4751	0.4419	7.67
XGBoost	0.8641	0.5768	0.5051	0.4677	28.41

```
from sklearn.metrics import classification_report

xgb_multi = multi_models["XGBoost"]
xgb_multi_predictions = xgb_multi.predict(X_test_known)
```